



CS240 Algorithm Design and Analysis

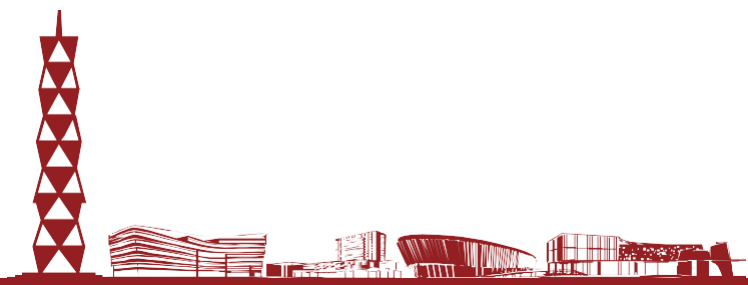
Lecture 7

Network Flow

Qingying Hao
Spring 2026
2026.03.24



Max-flow and Min-Cut





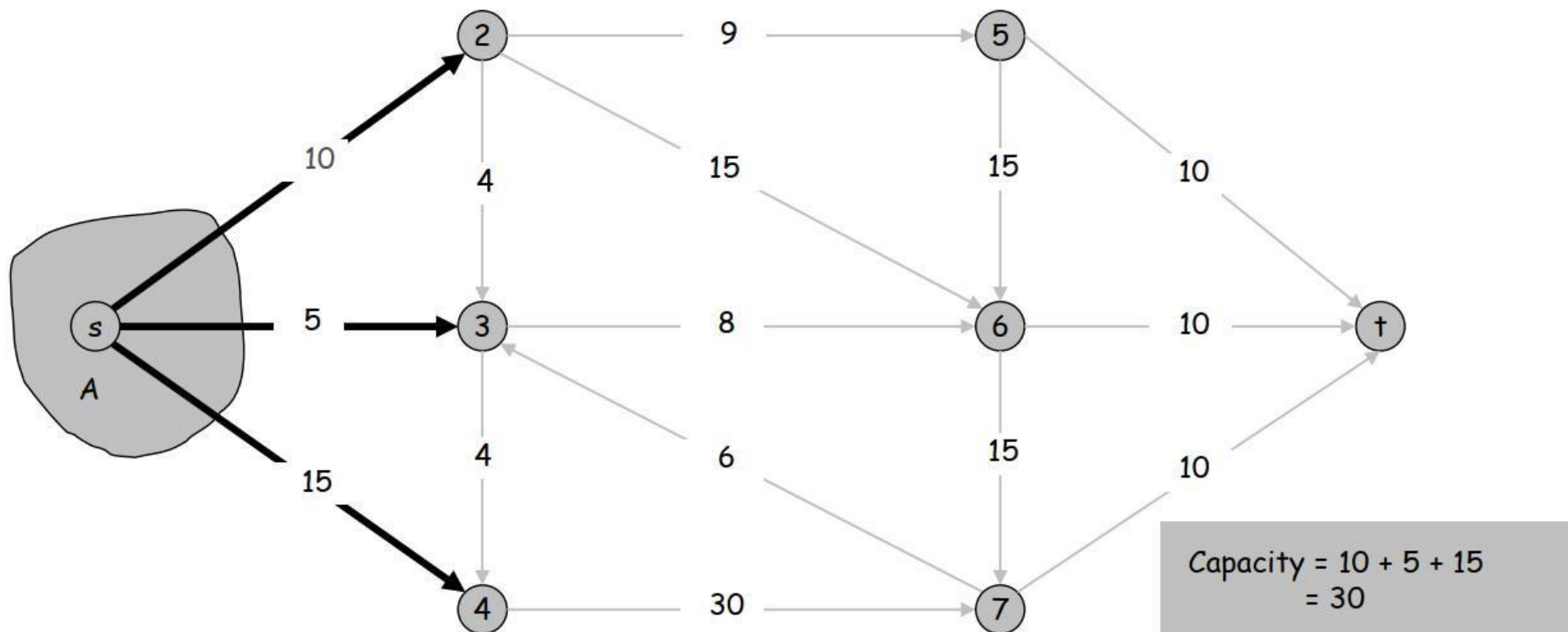
Cuts



- **Def.** An **s-t cut** is a partition (A, B) of V with $s \in A$ and $t \in B$

- **Def.** The **capacity** of a cut (A, B) is:

$$cap(A, B) = \sum_{e \text{ out of } A} c(e)$$





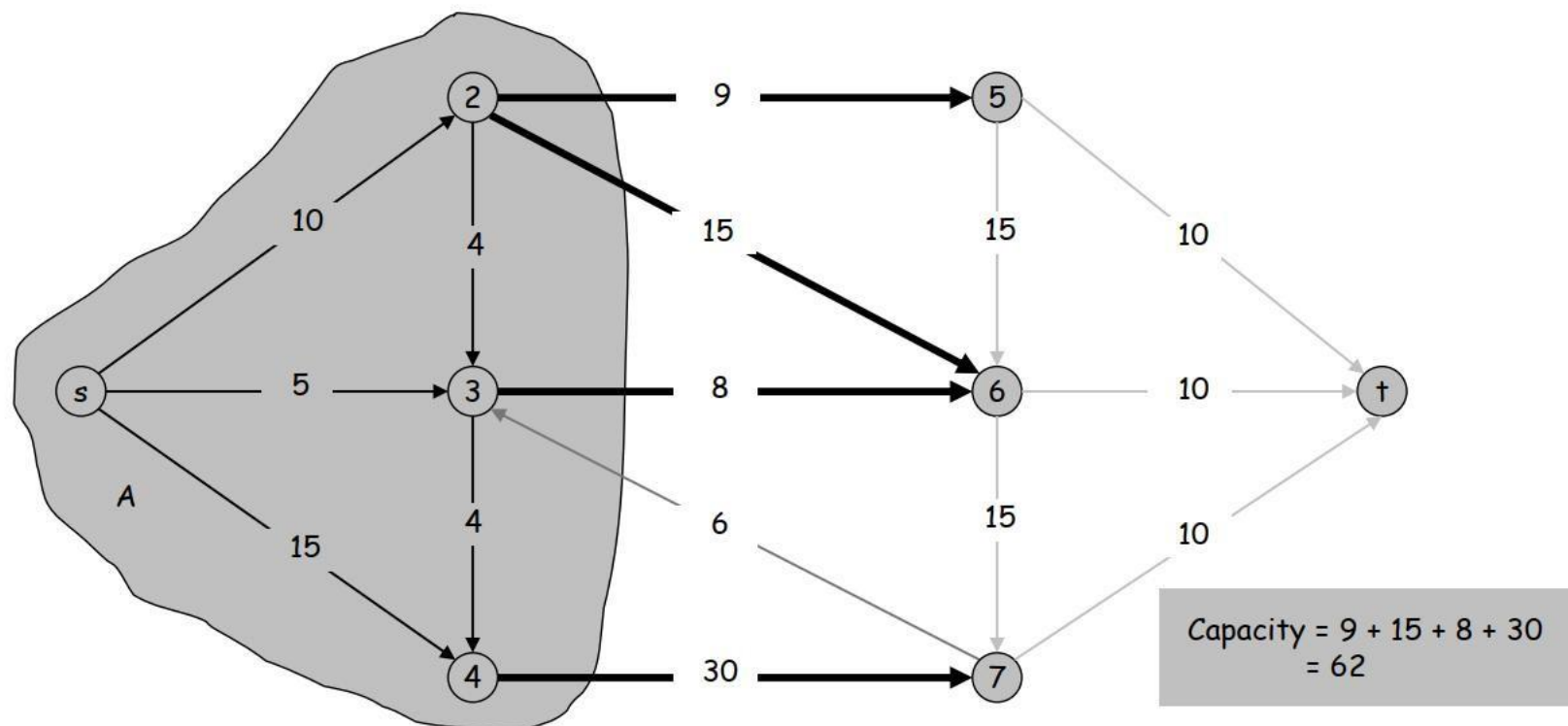
Cuts



- **Def.** An **s-t cut** is a partition (A, B) of V with $s \in A$ and $t \in B$

- **Def.** The **capacity** of a cut (A, B) is:

$$cap(A, B) = \sum_{e \text{ out of } A} c(e)$$

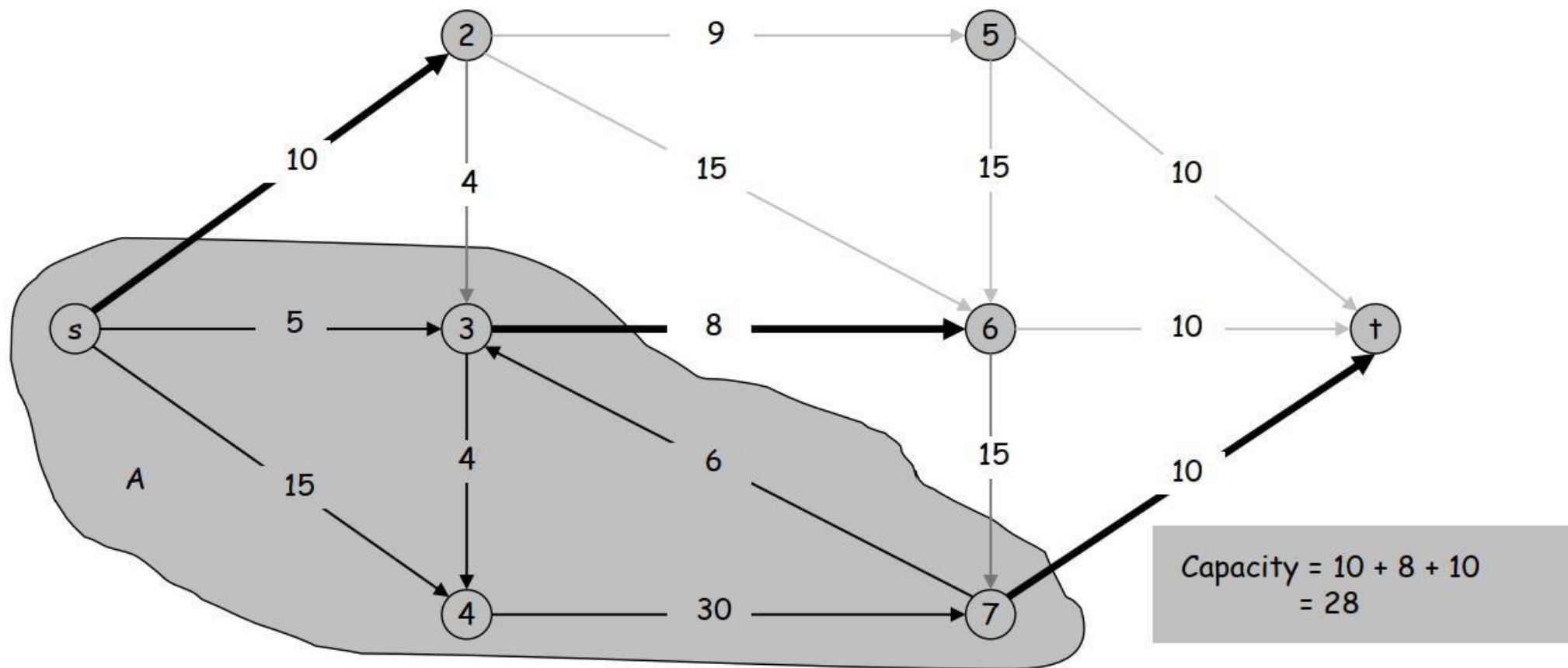




Minimum Cut Problem



- **Min s-t cut problem.** Find an s-t cut of minimum capacity



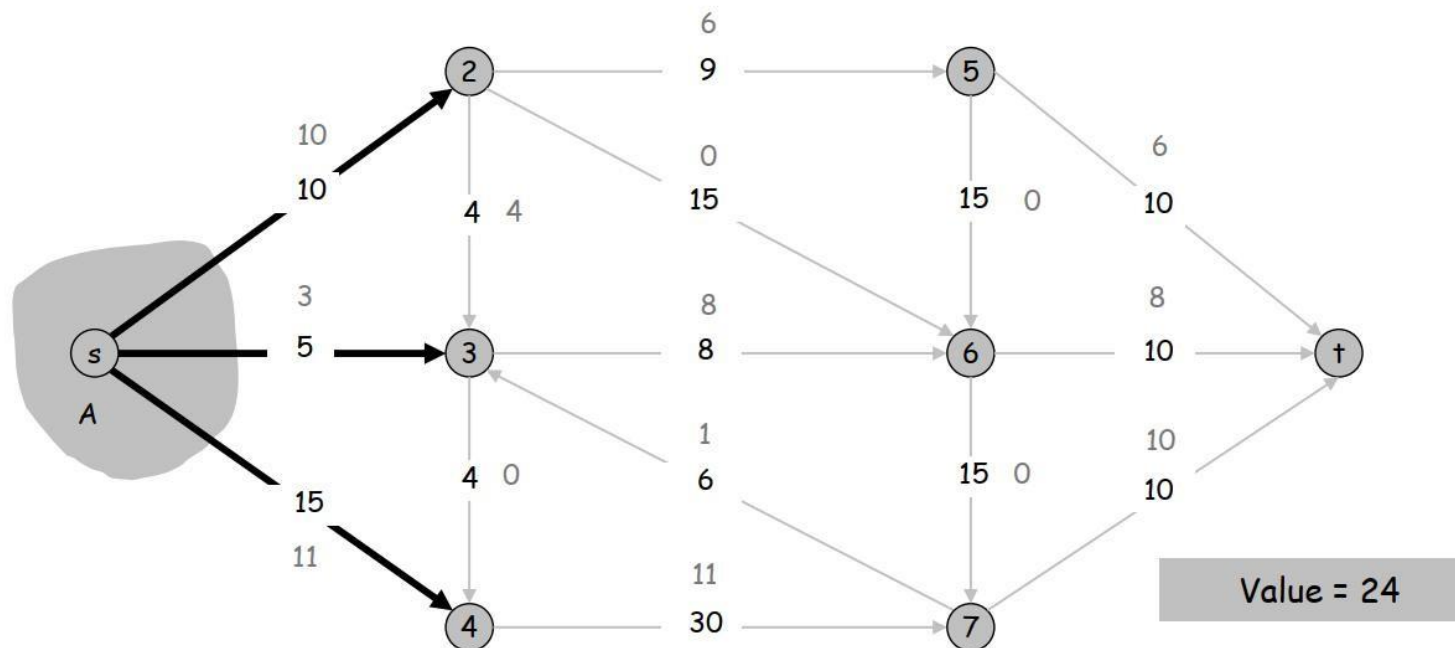


Flows and Cuts



- **Flow value lemma.** Let f be any flow, and let (A, B) be any s - t cut
- Then, the net flow sent across the cut is equal to the amount leaving s

$$\sum_{e \text{ out of } A} f(e) - \sum_{e \text{ in to } A} f(e) = v(f)$$



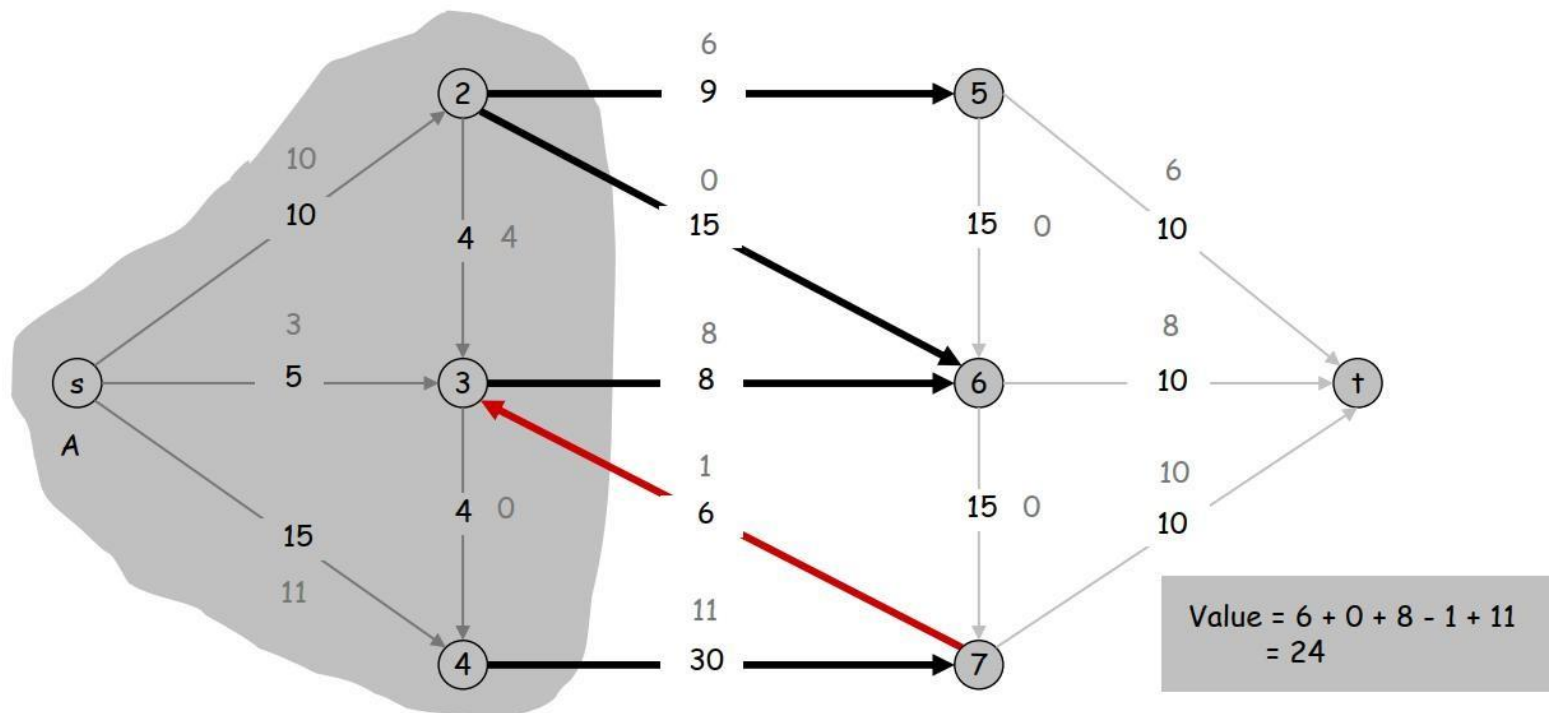


Flows and Cuts



- **Flow value lemma.** Let f be any flow, and let (A, B) be any s - t cut
- Then, the net flow sent across the cut is equal to the amount leaving s

$$\sum_{e \text{ out of } A} f(e) - \sum_{e \text{ in to } A} f(e) = v(f)$$

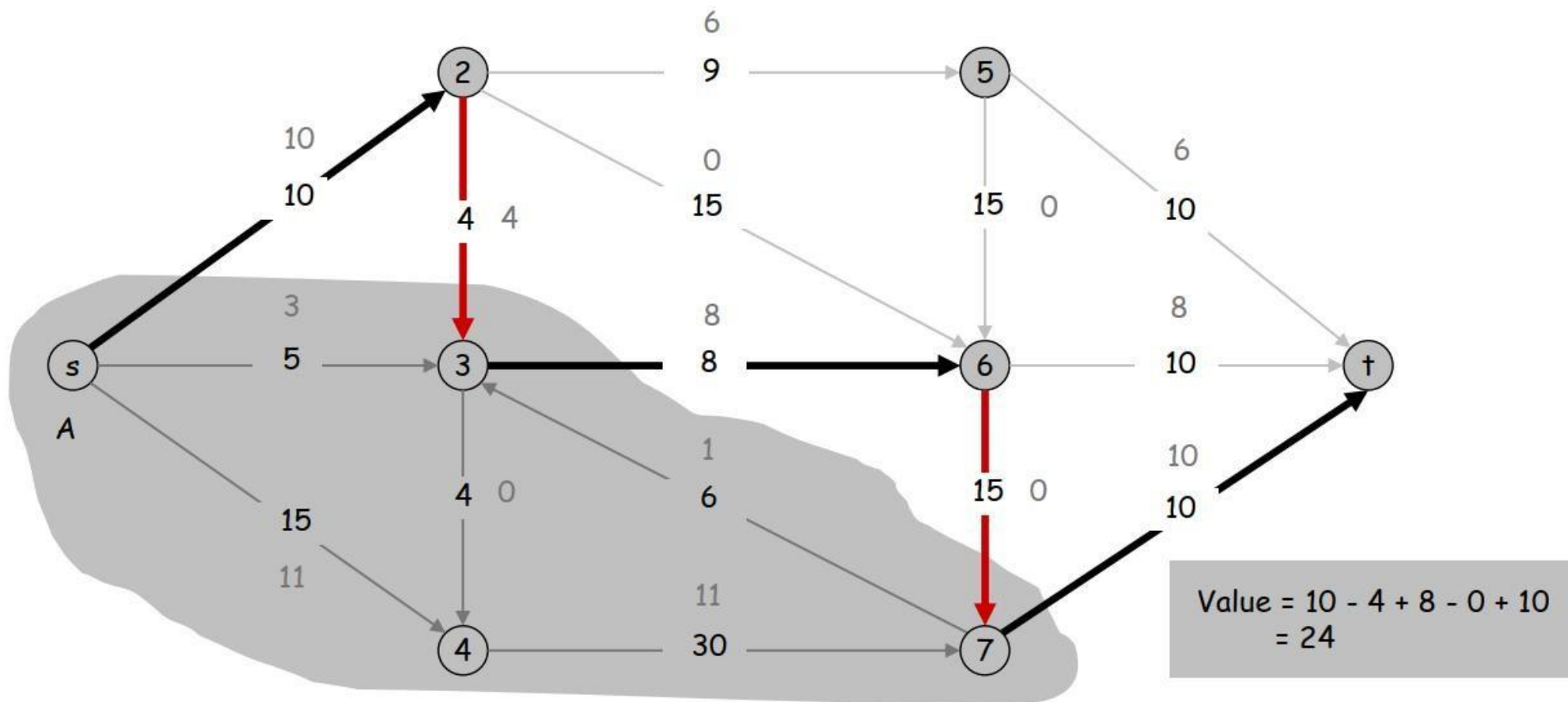




Flows and Cuts



- **Flow value lemma.** Let f be any flow, and let (A, B) be any s - t cut
- Then, the net flow sent across the cut is equal to the amount leaving s





- **Flow value lemma.** Let f be any flow, and let (A, B) be any s - t cut. Then,

$$\sum_{e \text{ out of } A} f(e) - \sum_{e \text{ in to } A} f(e) = v(f).$$

Pf.

$$v(f) = \sum_{e \text{ out of } s} f(e)$$

by flow conservation, all terms
except $v = s$ are 0

$$\sum_{e \text{ out of } v} f(e) - \sum_{e \text{ into } v} f(e) = 0$$

$$\rightarrow = \sum_{v \in A} \left(\sum_{e \text{ out of } v} f(e) - \sum_{e \text{ in to } v} f(e) \right)$$

$$= \sum_{e \text{ out of } A} f(e) - \sum_{e \text{ in to } A} f(e).$$



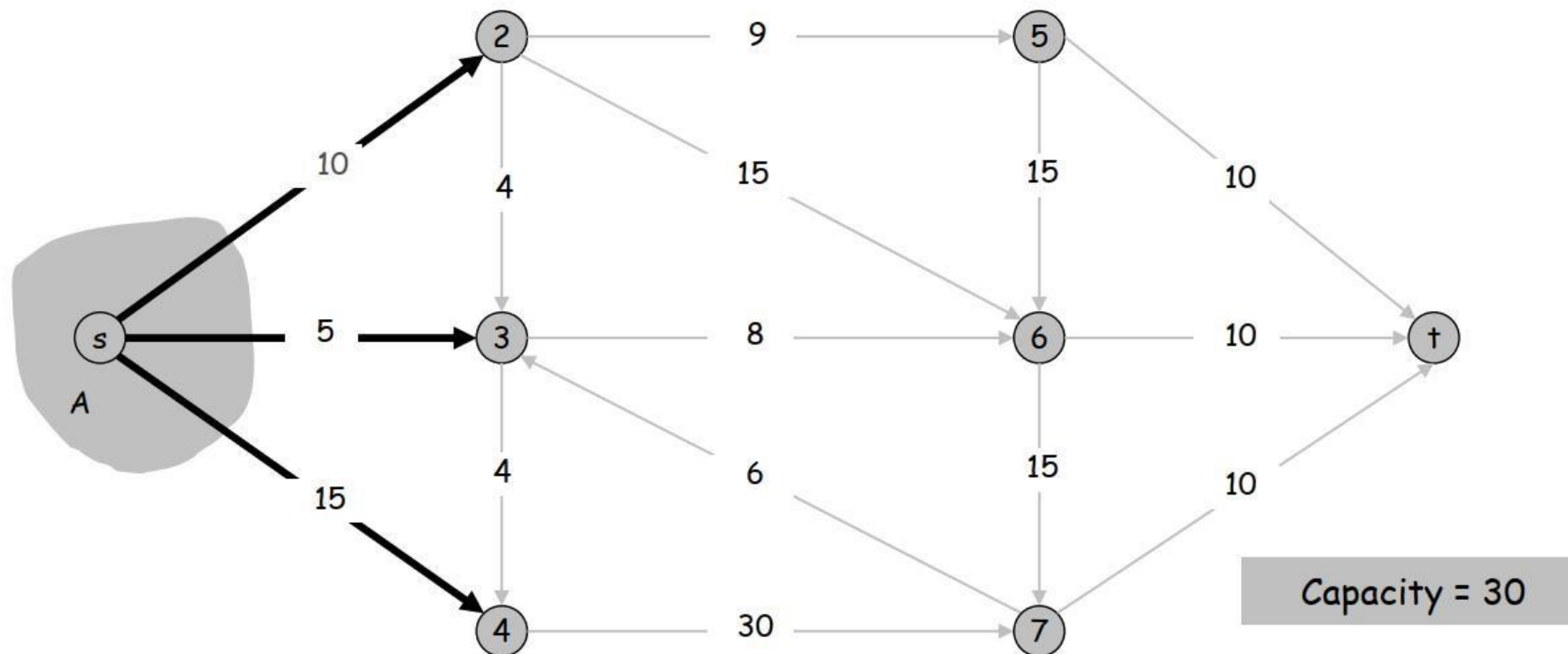


Flows and Cuts



- **Weak duality.** Let f be any flow, and let (A, B) be any s - t cut. Then the value of the flow is at most the capacity of the cut

Cut capacity = 30 $\Rightarrow$ Flow value ≤ 30





Flows and Cuts



- **Weak duality.** Let f be any flow. Then, for any s - t cut (A, B) we have $v(f) \leq \text{cap}(A, B)$
- Pf.

$$\begin{aligned} v(f) &= \sum_{e \text{ out of } A} f(e) - \sum_{e \text{ in to } A} f(e) \\ &\leq \sum_{e \text{ out of } A} f(e) \\ &\leq \sum_{e \text{ out of } A} c(e) \\ &= \text{cap}(A, B) \quad \square \end{aligned}$$





Max-Flow Min-Cut Theorem



- **Augmenting path theorem.** Flow f is a max flow iff there are no augmenting paths
- **Max-flow min-cut theorem.** [Ford-Fulkerson 1956] The value of the max flow is equal to the value of the min cut
- **Proof strategy.** We prove both simultaneously by showing the equivalence of the following three conditions for any flow f :
 - (i) There exists a cut (A, B) such that $v(f) = \text{cap}(A, B)$
 - (ii) Flow f is a max flow
 - (iii) There is no augmenting path relative to f
- (i) $\rightarrow$ (ii) This was the corollary to weak duality lemma
- (ii) $\rightarrow$ (iii) We show contrapositive
 - If there exists an augmenting path, then we can improve f by sending flow along path



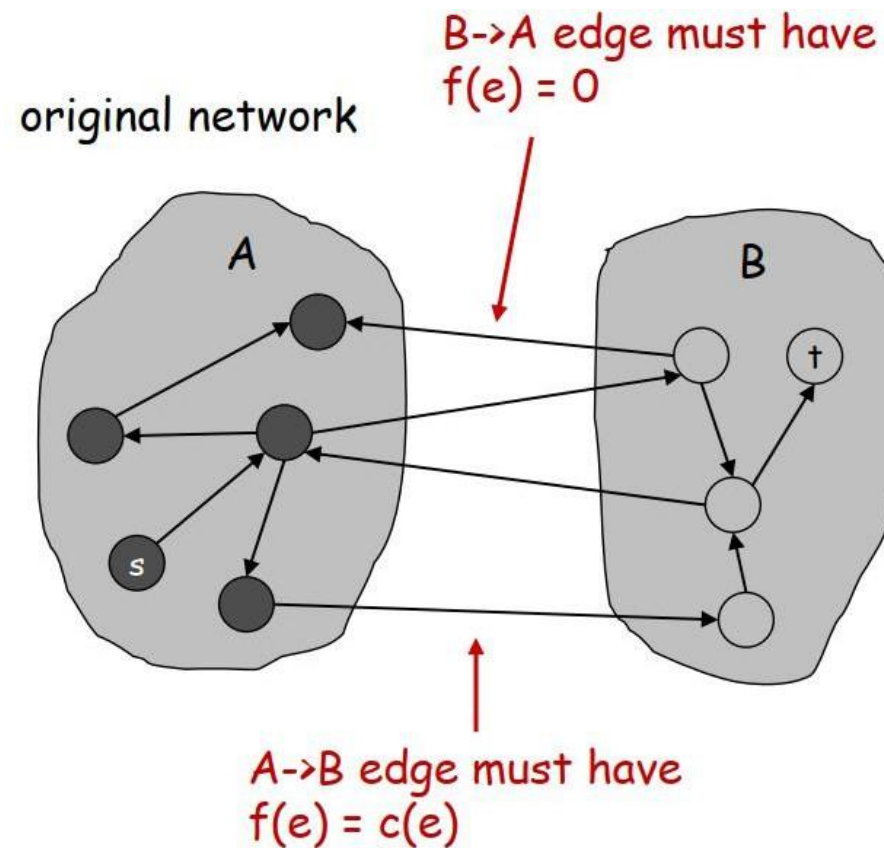


Proof of Max-Flow Min-Cut Theorem



- (iii) $\rightarrow$ (i)
 - Let f be a flow with no augmenting paths
 - Let A be set of vertices reachable from s in residual graph
 - By definition of A , $s \in A$
 - By definition of f , $t \notin A$

$$\begin{aligned} v(f) &= \sum_{e \text{ out of } A} f(e) - \sum_{e \text{ in to } A} f(e) \\ &= \sum_{e \text{ out of } A} c(e) \\ &= \text{cap}(A, B) \quad \blacksquare \end{aligned}$$





Max-Flow Min-Cut Theorem Summary



Step 1. Any cut gives an upper bound on the flow

Any flow from s to t must cross every s - t cut.

So for any cut (A, B) ,

$$|V(f)| \leq ca(A, B)$$

Step 2. The minimum cut gives the tightest upper bound

Among all possible cuts, the minimum cut has the smallest capacity. So:

$$|V(f)| \leq \min\text{-cut}$$

Step 3. If there is no augmenting path, the current flow is “stuck”

When there is no augmenting path in the residual graph:

all edges from A to B are saturated

all edges from B to A carry zero flow

So the flow exactly fills that cut:

$$|V(f)| = cap(A, B)$$

Step 4. Therefore, max flow = min cut





Choosing Good Augmenting Paths



Running Time

- **Assumption.** All capacities are integers between 1 and C
- **Invariant.** Every flow value $f(e)$ and every residual capacities $c_f(e)$ remains an integer throughout the algorithm
- **Integrality theorem.** If all capacities are integers, then there exists a max flow f for which every flow value $f(e)$ is an integer
 - Pf.** Since algorithm terminates, theorem follows from invariant
- **Theorem.** The algorithm terminates in at most $v(f^*)$ (value of max flow) $\leq nC$ iterations.
 - Pf.** Each augmentation increase value by at least 1
- **Corollary.** Running time of Ford-Fulkerson is $O(mnC)$ ← Polynomial?

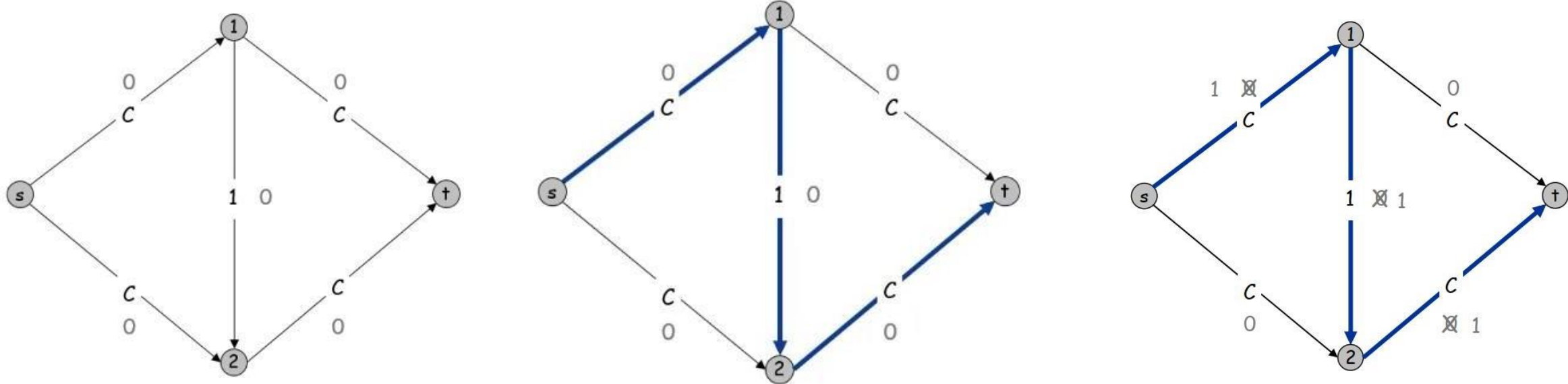


Ford-Fulkerson: Exponential Number of Augmentations

- **Generic Ford-Fulkerson algorithm is not polynomial in input size?**

$m, n,$ and $\log C$

- **An example:** If max capacity is C , then the algorithm can take $\geq C$ iterations



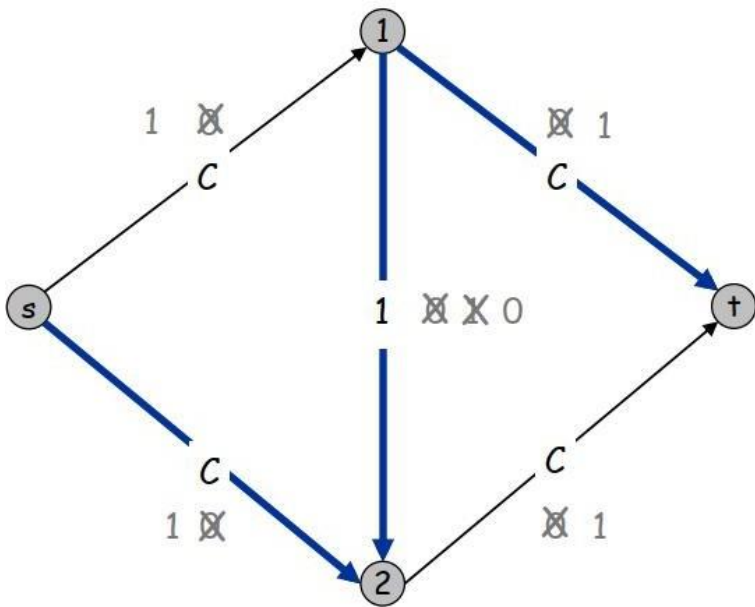
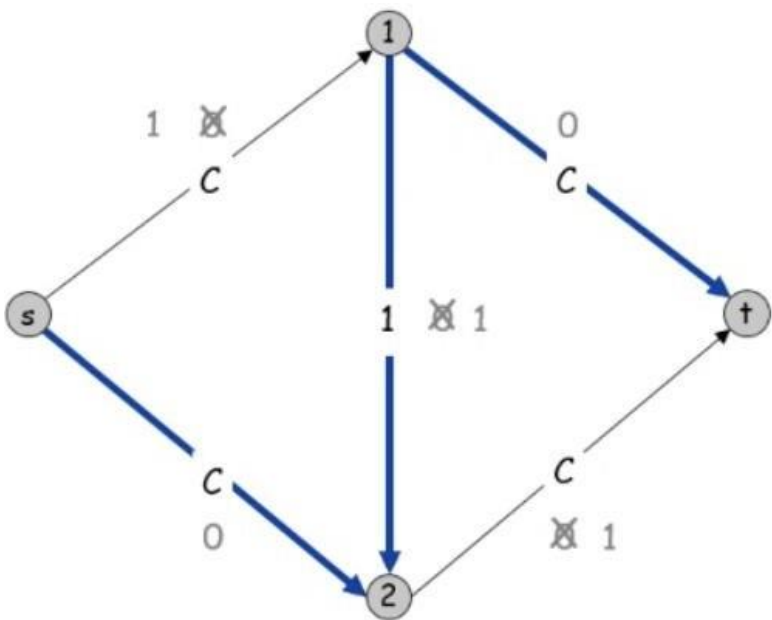


Ford-Fulkerson: Exponential Number of Augmentations

- **Generic Ford-Fulkerson algorithm is not polynomial in input size?**

$m, n,$ and $\log C$

- **An example:** If max capacity is C , then the algorithm can take $\geq C$ iterations



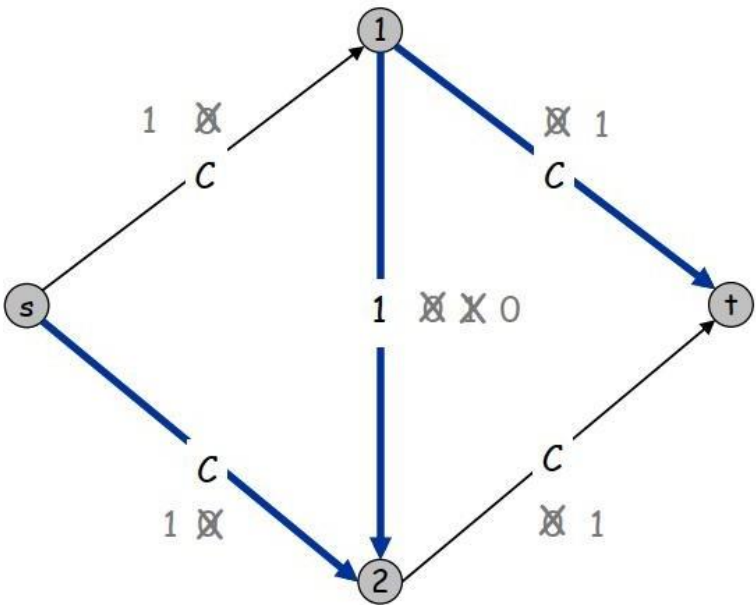


Ford-Fulkerson: Exponential Number of Augmentations

- **Generic Ford-Fulkerson algorithm is not polynomial in input size?**

$m, n,$ and $\log C$

- **An example:** If max capacity is C , then the algorithm can take $\geq C$ iterations



Each augmenting path sends only 1 unit of flow
(#augmenting paths = $2C$)



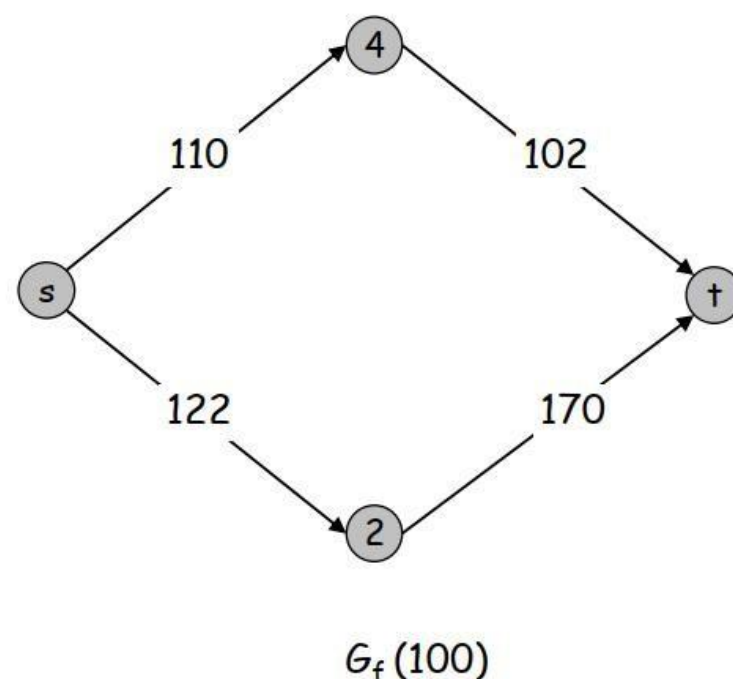
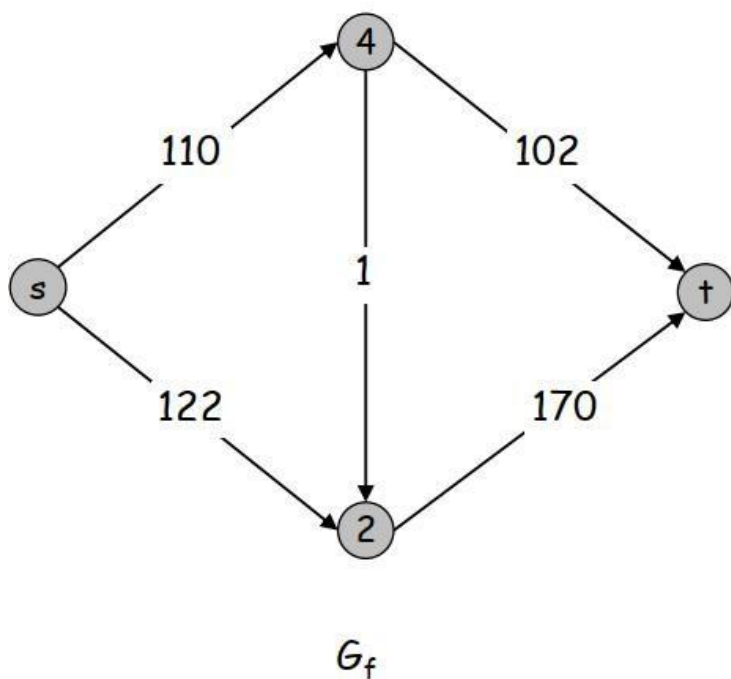
Choosing Good Augmenting Paths

- **Use care when selecting augmenting paths**
 - Some choices led to exponential algorithms
 - Clever choices lead to polynomial algorithms
 - (If capacities are irrational, algorithm not guaranteed to terminate!)
- **Goal: choose augmenting paths so that:**
 - Can find augmenting paths efficiently
 - Few iterations
- **Choose augmenting paths with: [Edmonds-Karp 1972, Dinitz 1970]**
 - Max bottleneck capacity
 - Fewest number of edges
 - **Sufficiently large bottleneck capacity**



Capacity Scaling

- **Intuition.** Choosing path with high bottleneck capacity
 - Maintain scaling parameter Δ
 - Let the **Δ -residual graph $G_f(\Delta)$** be the subgraph of the residual graph consisting of only arcs with capacity at least Δ





Capacity Scaling

```
Scaling-Max-Flow( $G, s, t, c$ ) {  
  foreach  $e \in E$   $f(e) \leftarrow 0$   
   $\Delta \leftarrow$  largest power of 2  $\leq C$   
  
  while ( $\Delta \geq 1$ ) {  
     $G_f(\Delta) \leftarrow \Delta$ -residual graph  
    while (there exists augmenting path  $P$  in  $G_f(\Delta)$ ) {  
       $f \leftarrow$  augment( $f, c, P$ )  
      update  $G_f(\Delta)$   
    }  
     $\Delta \leftarrow \Delta / 2$   
  }  
  return  $f$   
}
```



Capacity Scaling: Correctness

- **Assumption.** All edge capacities are integers between 1 and C
- **Integrality invariant.** All flow and residual capacity values are integral
- **Correctness.** If the algorithm terminates, then f is a max flow
- **Pf.**
 - By integrality invariant, when $\Delta = 1 \rightarrow G_f(\Delta) = G_f$
 - Upon termination of $\Delta = 1$ phase, there are no augmenting paths



Capacity Scaling: Running Time

- **Lemma 1.** The outer while loop repeats $1 + \lfloor \log_2 C \rfloor$ times
- Pf. Initially $C/2 < \Delta \leq C$. Δ decreases by a factor of 2 each iteration
- **Lemma 2.** Let f be the flow at the end of a Δ -scaling phase. Then the value of the maximum flow is at most $v(f) + m\Delta$ (proof on next slide)

$$v(f^*) \leq v(f) + m\Delta$$

- **Lemma 3.** There are at most $2m$ augmentations per scaling phase.
 - Let f be the flow at the end of the previous scaling phase
 - $L2 \rightarrow V(f^*) \leq v(f) + m(2\Delta)$
 - Each augmentation in a Δ -phase increases $v(f)$ by at least Δ
- **Theorem.** The scaling max-flow algorithm finds a max flow in $O(m \log C)$ augmentations. It can be implemented to run in $O(m^2 \log C)$ time

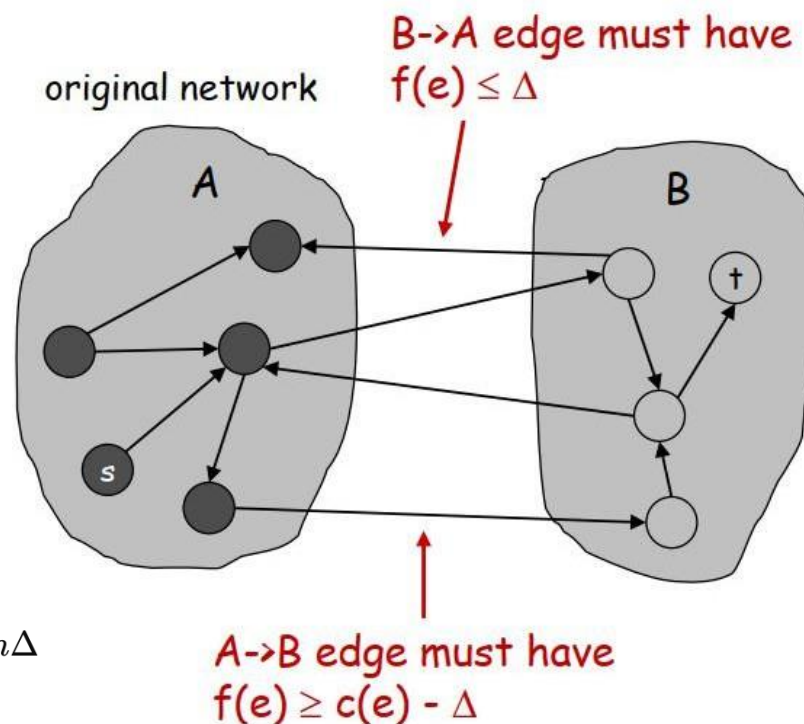
- $\Delta = 16$
- $\Delta = 8$
- $\Delta = 4$
- $\Delta = 2$
- $\Delta = 1$



Capacity Scaling: Running Time

- **Lemma 2.** Let f be the flow at the end of a Δ -scaling phase. Then the value of the maximum flow is at most $v(f) + m\Delta$
- **Pf.** (almost identical to proof of max-flow min-cut theorem)
 - We show that at the end of a Δ -phase, there exists a cut (A, B) such that $\text{cap}(A, B) \leq v(f) + m\Delta$
 - Choose A to be the set of nodes reachable from s in $G_f(\Delta)$
 - By definition of A , $s \in A$
 - By definition of f , $t \notin A$

$$\begin{aligned} v(f) &= \sum_{e \text{ out of } A} f(e) - \sum_{e \text{ in to } A} f(e) \\ &\geq \sum_{e \text{ out of } A} (c(e) - \Delta) - \sum_{e \text{ in to } A} \Delta \\ &= \sum_{e \text{ out of } A} c(e) - \underbrace{\sum_{e \text{ out of } A} \Delta - \sum_{e \text{ in to } A} \Delta}_{\sum_{e \text{ out of } A} \Delta + \sum_{e \text{ into } A} \Delta \leq m\Delta} \\ &\geq \text{cap}(A, B) - m\Delta \end{aligned}$$





Capacity Scaling: Running Time



- **Lemma 1.** The outer while loop repeats $1 + \lfloor \log_2 C \rfloor$ times
- Pf. Initially $C/2 < \Delta \leq C$. Δ decreases by a factor of 2 each iteration

- $\Delta = 16$
- $\Delta = 8$
- $\Delta = 4$
- $\Delta = 2$
- $\Delta = 1$

- **Lemma 2.** Let f be the flow at the end of a Δ -scaling phase. Then the value of the maximum flow is at most $v(f) + m\Delta$ (proof on next slide)

$$v(f^*) \leq v(f) + m\Delta$$

$$v(f^*) \leq \text{cap}(A, B) \leq v(f) + m\Delta$$

- **Lemma 3.** There are at most $2m$ augmentations per scaling phase.
 - Let f be the flow at the end of the previous scaling phase.
 - $L2 \rightarrow V(f^*) \leq v(f) + m(2\Delta)$
 - Each augmentation in a Δ -phase increases $v(f)$ by at least Δ

Previous round: 2Δ -phase
Now: Δ -phase

Each augmentation increases $v(f)$ by at least Δ

$$m(2\Delta) = 2m\Delta \quad \frac{2m\Delta}{\Delta} = 2m$$

- **Theorem.** The scaling max-flow algorithm finds a max flow in $O(m \log C)$ augmentations. It can be implemented to run in $O(m^2 \log C)$ time





Next Time: Network Flow (Cont.)